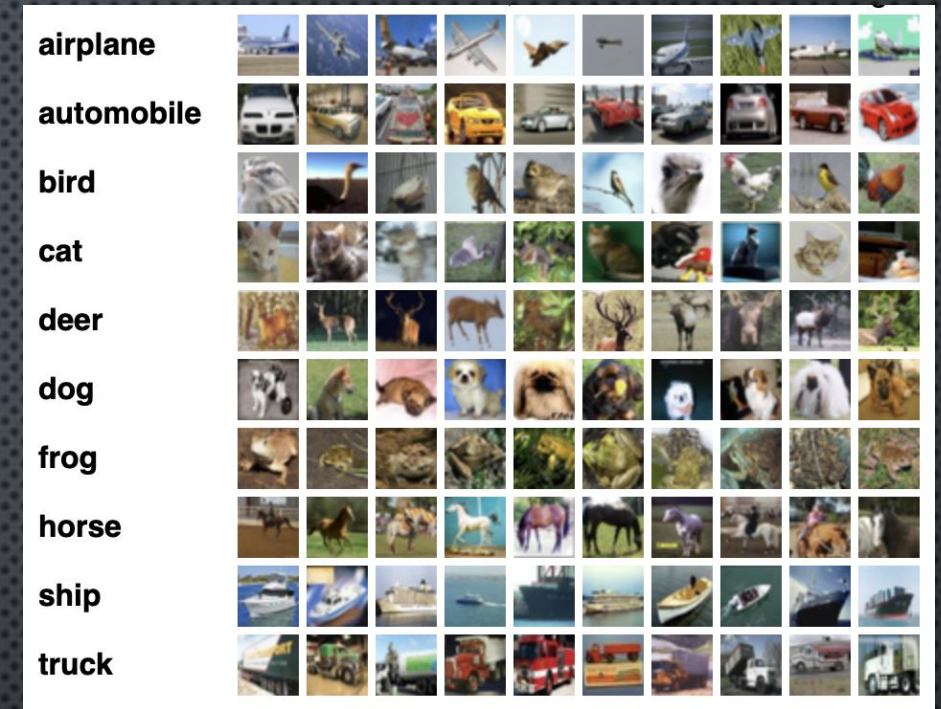


HOMEWORK 2 - CLASSIFICATORI

BENEDETTA NASSI 1000059819

CIFRAR-10

Il dataset in questione è CIFRAR-10, il quale contiene **60.000** immagini in formato **32x32**, da classificare in **9** **classi distinte**.



Il dataset è suddiviso in **cinque** batch di **addestramento** e un batch di **test**, ciascuno contenente 10.000 immagini:

- Il batch di **test** contiene 1.000 immagini per **ciascuna** classe.
- I batch di **addestramento** contengono le **restanti immagini** in ordine casuale, ma alcuni batch di addestramento possono contenere più immagini di una classe rispetto ad un'altra. Nel complesso, contengono esattamente 5.000 immagini per **ciascuna** classe.

CARICAMENTO DEL DATASET

Per sfruttare la divisione di default del dataset ho usato la funzione **`load_data()`**

```
from tensorflow.keras.datasets import cifar10
from sklearn.preprocessing import StandardScaler

(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

- **Sotto-campionamento:**

```
# Sottocampiono il 10% di CIFAR-10 training set
n_percent = 0.1
X_train, _, y_train, _ = train_test_split(
    X_train_full, y_train_full,
    train_size= n_percent,
    random_state=42
)
```

Poiché l'addestramento di ciascun modello sarebbe risultato **oneroso** in termini di costo computazionale, ho deciso di effettuare un **sotto-campionamento**.

Ho deciso di utilizzare solo il **10%** del dataset originale

MODEL SELECTION

Lo scopo della *model selection* è trovare la migliore combinazione di **iperparametri** per ciascun modello. Per tutti i modelli richiesti nella consegna ho deciso di effettuare la model selection tramite **grid-search**:

Perché la grid-search?

Sotto-campionamento



MASSIMO UTILIZZO DEI DATI DISPONIBILI: lavorando su una piccola porzione del dataset, grid-search implica la cross-validation, che consente di usare **tutti i dati sia per l'addestramento che per la valutazione**, a rotazione

MENO VARIANZA NELLA VALUTAZIONE: con pochi dati, una semplice divisione casuale in train/test può produrre risultati altamente variabili (grid-search riduce questa varianza)

Esaustiva



esplora in modo completo **tutte le combinazioni di iperparametri** specificate, garantendo una ricerca sistematica del set ottimale

Di seguito riporto una sintetica descrizione dei **parametri** su cui ho ritenuto opportuno eseguire la **grid-search**

- **Regressione logistica:**

L'efficienza della regressione logistica è fortemente influenzata dal **parametro C**, che rappresenta l'**inverso della forza di regolarizzazione** (λ):

- **Valori piccoli di C** → **Forte regolarizzazione** → Il modello è più semplice e tende a generalizzare meglio.
- **Valori grandi di C** → **Debole regolarizzazione** → Il modello è più complesso, più vicino all'overfitting.

- **SVM:**

Per lavorare su questo dataset ho pensato che usare il **kernel gaussiano** ('rbf') fosse una buona scelta, poiché molto flessibile, capace di gestire separazioni altamente non lineari.

Ho quindi eseguito la model selection per determinare i migliori valori di C: un **parametro di regolarizzazione** che controlla il **compromesso** tra ottenere un **marginale ampio** e **classificare correttamente** i punti di addestramento.

- **C grande** → **meno tolleranza agli errori** → margine più stretto → rischio di **overfitting**
- **C piccolo** → **più tolleranza agli errori** → margine più ampio e un modello più semplice → rischio di **underfitting**

- **k-NN**

Il parametro '**n_neighbors**' in **K-NN** rappresenta il numero **k** di vicini più prossimi che il modello considera per fare una predizione:

- K piccoli → può **overfittare**
- K grandi → rischio di **underfit**

'weight_options' in **K-Nearest Neighbors** controlla come pesare l'influenza dei vicini nella predizione:

- 'uniform': tutti i vicini contano allo stesso modo, indipendentemente da quanto sono lontani.
- 'weighted': I vicini più vicini pesano di più (di solito il peso è proporzionale a $1/d$, dove d è la distanza).

- **Decision tree**

'max_depth' imposta la profondità massima dell'albero:

- Valori piccoli: L'albero è poco profondo, più semplice → rischia **underfitting**
- Valori grandi: L'albero è più profondo, più flessibile → rischia **overfitting**

'min_samples_split' è il numero minimo di campioni richiesto in un nodo per poter essere diviso:

- **Valori piccoli**: l'albero può suddividere molto → può **overfittare**
- **Valori grandi**: richiede più dati per ogni split → più **semplice**, rischia **underfitting**

RISULTATI

Dai risultati ottenuti risulta evidente che nessuno di questi modelli si presti a risolvere in maniera efficace questo problema di classificazione, in quanto le migliori prestazioni riscontrate (quelle di SVM) garantiscono un'accuratezza pari circa al 45%

- **Regressione Logistica:**

```
Miglior modello Logistic Regression: {'C': 0.1}  
Accuracy (CV): 0.305  
Accuracy: 0.3089  
Precision: 0.30935903995994746  
Recall: 0.3089  
F1: 0.30868830962675153
```

- **K-NN:**

```
Miglior modello KNN: {'n_neighbors': 9, 'weights': 'distance'}  
Accuracy (CV): 0.2966  
Accuracy: 0.2899  
Precision: 0.40002872675835055  
Recall: 0.2899  
F1: 0.2634938437198665
```

- **SVM:**

```
Miglior modello SVM: {'C': 10}  
Accuracy (CV): 0.4488  
Accuracy: 0.4508  
Precision: 0.4526041824605753  
Recall: 0.4508  
F1: 0.4507067901870715
```

- **Decision tree:**

```
Miglior modello Decision Tree: {'max_depth': 7, 'min_samples_split': 10}  
Accuracy (CV): 0.2552  
Accuracy: 0.2487  
Precision: 0.24445470326051935  
Recall: 0.2487  
F1: 0.24217789692811492
```